```cpp
// Question 1:

#include<bits/stdc++.h>
using namespace std;

int solve(int n) {
    if (n > 100){
        return n - 10;
    }
    return solve(solve(n + 11));
}

int main() {
    int n;
    cin>>n;

    cout<<solve(n);

    return 0;
}
```

```cpp
// Question 2:

#include<bits/stdc++.h>
using namespace std;

int maxDistance(vector<int>& position, int m) {
    sort(position.begin(), position.end());
    int n = position.size();

    int low = 1, high = position[n - 1] - position[0];
    int ans = 0;

    while (low <= high) {
        int mid = (low + high) / 2;

        int balls = 1;
        int prev = position[0];

        for (int i = 1; i < n; i++) {
            if (position[i] - prev >= mid) {
                balls++;
                prev = position[i];
            }
        }

        if (balls >= m) {
            ans = mid;
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }

    return ans;
}

int main() {
    int n;
    cout<<"Enter size: ";
    cin >> n;
    cout<<"Enter array: ";
    vector<int> position(n);
    for (int i = 0; i < n; i++) {
        cin >> position[i];
    }

    int m;
    cout<<"m: ";
    cin >> m;
    cout<<maxDistance(position, m);

    return 0;
}
```

```cpp
// Question 3:

#include<bits/stdc++.h>
using namespace std;

int solve(string s, int k) {
    unordered_map<char, int> mp;
    int left = 0, maxCount = 0, ans = 0;

    for (int right = 0; right < s.size(); right++) {
        mp[s[right]]++;
        maxCount = max(maxCount, mp[s[right]]);

        if ((right - left + 1) - maxCount > k) {
            mp[s[left]]--;
            left++;
        }

        ans = max(ans, right - left + 1);
    }

    return ans;
}

int main() {
    string s;
    int k;
    cin>>s>>k;

    cout<<solve(s, k);

    return 0;
}
```